



Inżynieria Oprogramowania

dr Katarzyna Grzesiak-Kopeć



Warunki zaliczenia wykładu



<https://sylabus.uj.edu.pl/pl/document/0bf58b2d-929c-4801-bb1e-1ee0d9e1c120.pdf>

Rodzaj zajęć	Formy zaliczenia	Warunki zaliczenia przedmiotu
wykład	zaliczenie pisemne	uzyskanie zaliczenia z ćwiczeń oraz uzyskanie co najmniej 50% punktów z testu zaliczeniowego; wykład prowadzony zdalnie
ćwiczenia	zaliczenie pisemne, projekt	obecność na zajęciach, zaliczenie projektu grupowego, uzyskanie co najmniej 50% punktów z każdego kolokwium; ćwiczenia mogą być prowadzony zdalnie albo/i stacjonarnie, szczegółowy harmonogram ustalany na początku semestru

Jako ocena końcowa zostanie przepisana ocena zaliczeniowa z ćwiczeń.

Plan wykładu

01

**Tworzenie
programów**

02

**Najlepsze
praktyki**

03

Wymagania

04

OOAD & UML

05

Techniki IO

06

Refaktoryzacja



Plan wykładu

07

**Mierzenie
programów**

08

**Ocena
jakości**

09

**Programowanie
strukturalne**

10

**Modelowanie
analityczne**

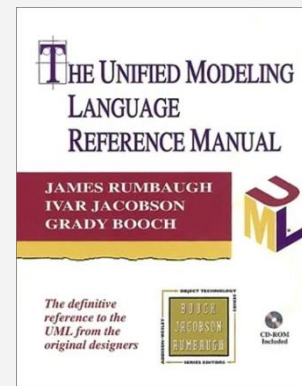
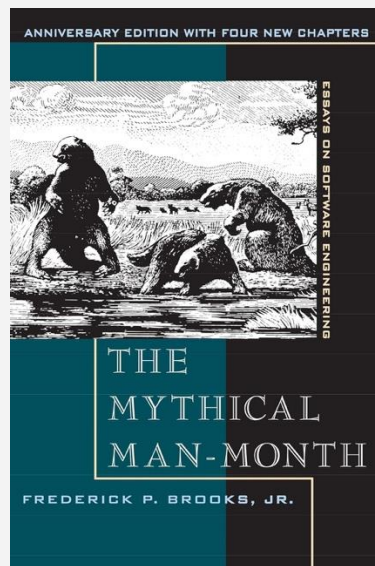
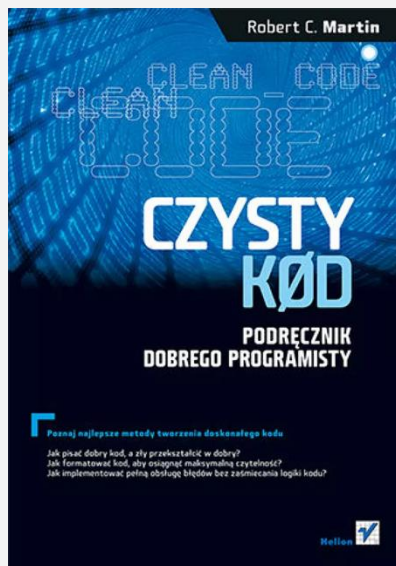
11

Testowanie



Literatura

1975





01

Tworzenie oprogramowania

Oprogramowanie jako produkt



Byt logiczny

Nie „zużywa się” jak byt fizyczny

Żyje – ciągła zmiana wymagań

Złożony produkt i proces

Oprogramowanie jako produkt



Składa się z:

- programów wykonywanych na komputerach każdej wielkości i rodzaju
- dokumentów elektronicznych i drukowanych
- danych

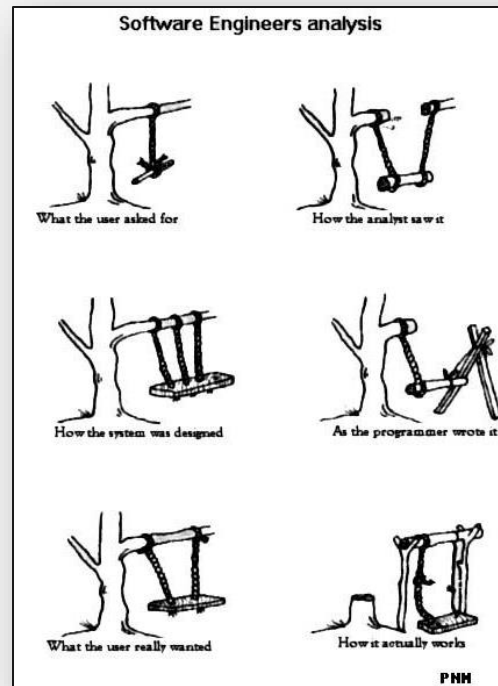
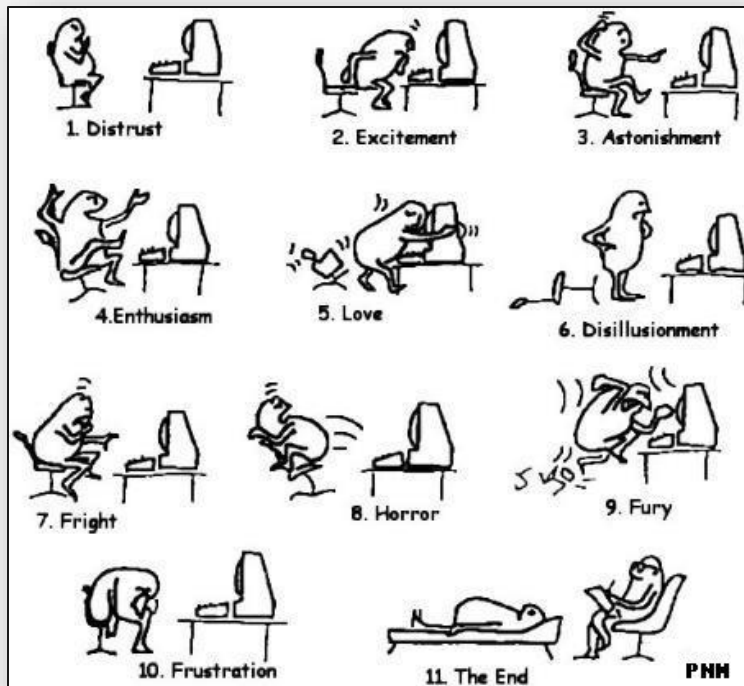


Inżynieria oprogramowania

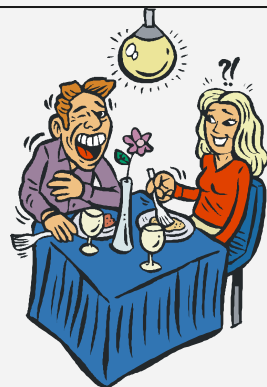
- 01** Zastosowanie systematycznego, zdyscyplinowanego, poddającego się ocenie ilościowej podejścia do wytwarzania, stosowania i pielęgnacji oprogramowania, czyli wykorzystanie tradycyjnej inżynierii w informatyce.
- 02** Dziedzina wiedzy zajmująca się badaniem metod jak w (1).



IO w praktyce



- **Tworzenie oprogramowania**



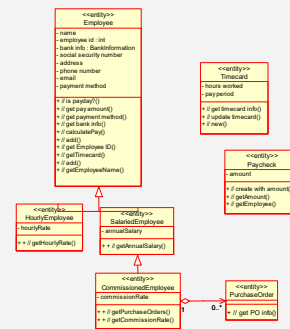
```
public Vector getLinesFromTriangles(Vector points, Vector lines){
    if(points == null) return null;

    planes = new Vector<>();
    pointInLines = new HashSet<>();
    lineInPoints = new HashSet<>();

    pointIn2D.double p = null;
    Line line = null;
    pointIn2D.double[] linePoints = null;
    boolean[] isOnLine = null;

    for(int i = 0; i < points.size(); i++){
        p = pointIn2D.double(points.get(i));
        isOnLine = new boolean[lines.size()];

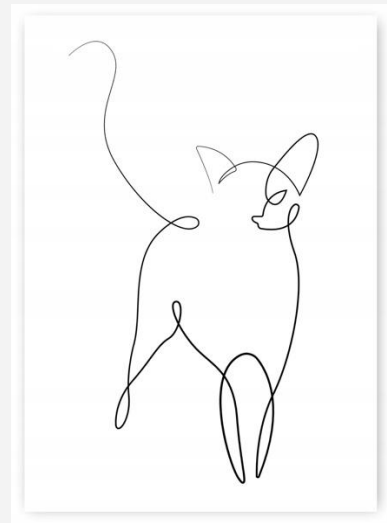
        for(int j = 0; j < lines.size(); j++){
            linePoints = pointIn2D.double(lines.get(j));
            line = new Line(linePoints[0], linePoints[1]);
            isOnLine[j] = line.isOnLine(p);
            addLineToLine(p, linePoints);
        }
        //if on line
        pointInLines.put(p, isOnLine);
    }
    //for i
```





Co to jest model?

Pewne uproszczenie rzeczywistości



Dlaczego modelujemy?



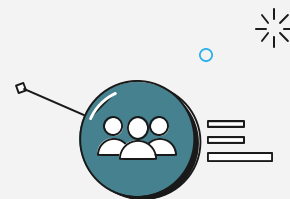
Lepiej zrozumieć system, który rozwijamy



Pokazanie zależności między elementami



Budujemy modele skomplikowanych systemów



Dlaczego modelujemy?



Pomaga w wizualizacji



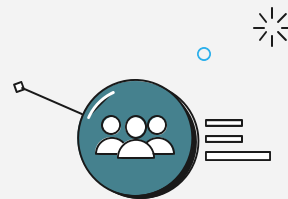
Pokazanie struktury i zachowania



Wzorzec wyznaczający kolejne kroki



Dokumentacja podjętych decyzji



Przeprowadzka



Jedna z najbardziej stresujących sytuacji

Syndrom pierwszych dni: gdzie to jest?

Jak uniknąć syndromu: mieć plan, wynająć kogoś kto ma plan

Nowy projekt to przeprowadzka



Nowa dziedzina i nowe wymagania

Nowa technologia

Nowy klient i współpracownicy

Jak rozmawiać z klientem?

Jak opisać projekt?

Co i jak dokumentować?

Jak zweryfikować?

...

Projekt vs przeprowadzka

Podobieństwa

Nie zbieramy doświadczeń = rewolucja
Nie umiemy oszacować terminu zakończenia
Zmiany w trakcie to katastrofa
Za późno identyfikujemy błędy



Różnice

Złożoność problemu
Zazwyczaj przeprowadzki są rzadsze

Jak ocenić projekt



Kiedy osiągnęliśmy sukces

Zmieściliśmy się w zaplanowanym budżecie

Zmieściliśmy się w zaplanowanym czasie

Spełniliśmy **rzeczywiste** wymagania klienta



Chaos Report Standish Group (od 1994)

Jaka liczba projektów kończy się sukcesem i dlaczego?

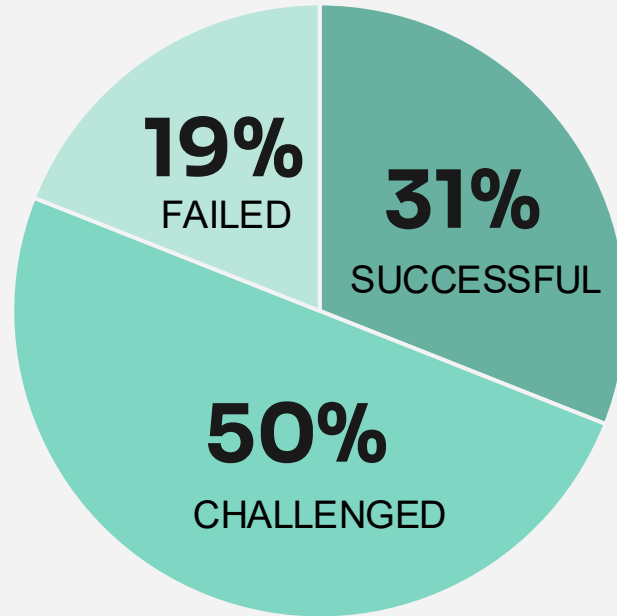
Chaos Report Standish Group (2015)

MODERN RESOLUTION FOR ALL PROJECTS

	2011	2012	2013	2014	2015
SUCCESSFUL	29%	27%	31%	28%	29%
CHALLENGED	49%	56%	50%	55%	52%
FAILED	22%	17%	19%	17%	19%

The Modern Resolution (OnTime, OnBudget, with a satisfactory result) of all software projects from FY2011 - 2015 within the new CHAOS database. Please note that for the rest of this report CHAOS Resolution will refer to the Modern Resolution definition not the Traditional Resolution definition.

Chaos Report Standish Group (2020)



Co wpływa na sukces projektu?

01 Wsparcie kadry zarządzającej

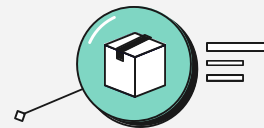
02 Dojrzałość emocjonalna

03 Zaangażowanie użytkownika

04 Optymalizacja

05 Wykwalifikowany personel

06 SAME



07 Biegła znajomość Agile

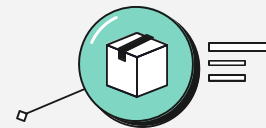
08 Umiarkowany stopień złożoności

09 Doświadczenie w zarządzaniu projektami

10 Jasne cele biznesowe

Co wpływa na sukces projektu 2020?

11 Dobre środowisko pracy



12 Dobry zespół

13 Dobry sponsor

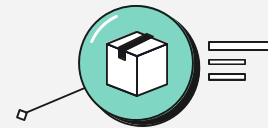
AGILE

Jest najskuteczniejsze

Review Standish Group – CHAOS 2020: Beyond Infinity

1960 – 1980

The Wild West



1980 – 2000

The Waterfall Period

2000 – 2024 –

The Agile Period

? –

The Infinite Flow Period

Uwaga na objawy

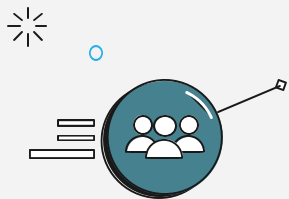
Niewyspani programiści

Pizza i cola pod drzwi



Brak powtarzalności procesu

Ile tym razem przekroczony budżet



Stres na koniec projektu

Oj, to nie tak miało być...

Marsz śmierci

Musicie to zrobić dwa razy szybciej niż konkurencja

Co robić?



Kontroluj: złożony problem; uwaga – język naturalny nas zawodzi



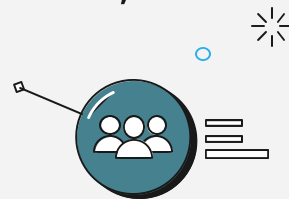
Dziel i zwyciężaj: dobre części nie zawsze dadzą działającą całość, muszą być niezależne, 7 ± 2 , hierarchie



Od rozmycia do skupienia: model



Dokumentacja: plany testów, założenia, to nie powieść



Co robić?



Nie przesadzaj z IO: lepsze jest wrogiem dobrego,
nie zaskakuj mile klienta



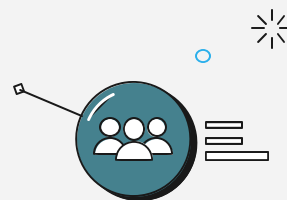
Przygotuj się na zmiany



Nie wynajduj koła



Przyjmij odpowiedzialność



Na pomoc!



Internet i literatura



Potrzebny nam proces?



Ocena procesu i standardy IO



Najlepsze praktyki



Metodyka (proces)

Definiuje **KTO** robi **CO**, **KIEDY** oraz **JAK**, żeby osiągnąć zamierzony cel.

Nowe lub
zmienione
wymagania

PROCES IO

Nowy lub
zmieniony
system

METODYKA



Można kontrolować



Można mierzyć



Można zmieniać i poprawiać



Orientacja na PRODUKT



Standardy IO



IEEE: The Institute of Electrical and Electronics Engineers



ISO: International Organization for Standardization



DoD: the US Department of Defense



Ocena procesu IO: Capability Maturity Model



Software Engineering Institute (SEI)



Pięciostopniowa miara dojrzałości organizacji do produkcji oprogramowania



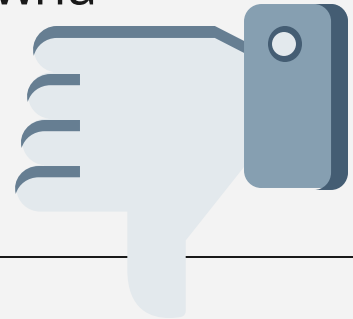
Ocena zakłada, że zawsze stosujemy te same procedury



Brak procedury oceny CMM – jest subiektywna



2003 zmierzch modelu



Recepty na dobry proces



technika wodospadu, model spiralny, model oparty na metodach formalnych, prototypowanie, Rapid Application Development (RAD), Component Based Development (CBD), Concurrent Development, Disciplined Software Development, Aspect-Oriented Software Development, Agile Process Models, The Rational Unified Process (RUP)...



Koniec!